

1 Problem Overview

Given a string s of n digits, we need to insert exactly p plus signs ('+') and q minus signs ('-') between consecutive digits to form a mathematical expression. One contiguous block of $lg = n - p - q$ digits will have no operator between them (they form a single number). The goal is to maximize the value of the resulting expression.

2 Key Observations

The crucial insight is that we have complete freedom in choosing:

1. Which contiguous block of lg digits stays together as a single number (this number will be added to the total).
2. Which of the remaining digits get '+' signs and which get '-' signs.

To maximize the expression, we need to make three strategic choices:

- **Choose the lexicographically largest block** of length lg from s to form the unoperated number. Since this number is added to the result, it should be as large as possible.
- **Assign the smallest digits** to be subtracted (with '-') — subtracting a small number reduces the total the least.
- **Assign the largest remaining digits** to be added (with '+') — adding large numbers increases the total the most.

3 Algorithm

The algorithm proceeds as follows:

1. Find the starting position **imax** of the lexicographically maximum subsequence of length lg in s .
2. Extract all digits **not** in this maximum block into array a .
3. Sort a in ascending order.
4. Subtract the first q smallest digits from the running sum.
5. Add the remaining p digits to the running sum.
6. Compute the final result by adding or subtracting the maximum block to/from this sum.

4 Correctness Proof

We prove that the algorithm always produces the maximum possible expression value.

- **Lemma 1.** The unoperated block should be the lexicographically maximum subsequence of length lg to maximize the expression value.

Proof. This block is added to the final result (unless it starts at position 0, where no operator precedes it). Among all contiguous blocks of length lg , the lexicographically maximum one has the largest possible numerical value. Therefore, choosing it maximizes the total. $\square$

- **Lemma 2.** In an optimal assignment, the q digits assigned minus signs should be the q smallest digits among those not in the maximum block.

Proof. Suppose some smaller digit $x < y$ where x has a '+' sign and y has a '-' sign. Swapping these signs changes the result by $(y - x) > 0$. Since we want to maximize the result, this swap is always beneficial. By repeatedly applying such swaps, we obtain an optimal solution where all q minus signs are assigned to the q smallest available digits. $\square$

- **Lemma 3.** Given Lemma 2, assigning '+' signs to the remaining p largest digits maximizes the expression.

Proof. With Lemma 2 satisfied, all other digits must have '+' signs. For any two such digits $x < y$ where x is added and y is not, swapping their roles increases the result by $(y - x) > 0$. Repeating this argument shows that the p largest among the remaining digits should receive '+' signs. $\square$

Theorem. The algorithm described produces an expression with maximum possible value.

Proof. By Lemma 1, the algorithm selects the lexicographically maximum block, which is optimal for the unoperated number. By Lemmas 2 and 3, sorting the remaining digits and assigning the smallest q to subtraction and largest p to addition is optimal. Since these decisions are independent, their combination yields a globally optimal solution. $\square$

5 Implementation Details

```
#define LGMAX 1002

typedef int NrMare[LGMAX];
NrMare secv, sum, rez;
```

- **NrMare** is a big integer type storing digits in little-endian order (least significant digit at index 0).

- The lexicographic comparison iterates through the string comparing characters at matching positions.
- Addition and subtraction functions handle arbitrary-length integers via array manipulation.
- The result sign is determined by comparing absolute values when subtraction is required.

The comparison function for substrings:

```
int compars(int i, int j)
//returns 1 if substring starting at i > substring starting
    at j
//returns 0 if equal, returns -1 otherwise
{
    int k;
    for (k = 0; k < lg && s[i+k] == s[j+k]; k++);
    if (k == lg) return 0;
    if (s[i+k] > s[j+k]) return 1;
    return -1;
}
```

The sorting and assignment step:

```
sort(a, a + nr);
for (i = 0; i < q; i++) suma = suma - (a[i] - '0');
for (i = q; i < nr; i++) suma = suma + (a[i] - '0');
```

Finally, the maximum block is added to or subtracted from the sum of other digits using the `adun` (add) and `dif` (subtract) functions.

6 Complexity Analysis

- Finding the maximum substring: $O(n \cdot lg) = O(n \cdot (n - p - q)) \leq O(n^2)$.
- Sorting the remaining $nr = n - lg$ digits: $O(n \log n)$.
- Big integer operations: $O(lg)$ or $O(n)$ per operation.

With $n \leq 1000$, this is easily within acceptable limits.